



**Dr. A. Q. Khan Institute of Computer
Science and
Information Technology**
Department of Computer Science



Lab REPORT

AI-Based Application Development in .NET

Submitted By
MUHAMMAD Usama Bilal

Reg NO#
232201097

Practical Assignment Report

AI-Based Mini Project (.NET)

1. Project Introduction

◆ Problem Statement

In today's digital world, users require quick and intelligent responses to their queries. Traditional systems lack automation and intelligence, making user interaction slow and inefficient.

◆ Objective

The objective of this project is to develop an **AI-based chatbot desktop application** using .NET that can:

- Interact with users in real time
 - Provide intelligent responses using AI API
 - Improve user experience through automation
-

2. System Design

◆ Architecture Diagram

Flow:

User → Windows Forms UI → API Request → Groq AI Model → Response → UI Display

◆ Workflow

1. User enters a message in the input box
 2. Application sends request to Groq API
 3. API processes the request using AI model
 4. AI generates response
 5. Response is displayed in chat window
-

3. Tools & Technologies

--	--	--	--	--

Tool/Technology

Purpose

.NET (Windows Forms) Desktop application development

C# Programming language

Groq API AI chatbot response generation

Newtonsoft.Json JSON parsing

HttpClient API communication

Docker Containerization

4. Code Snippets

API Integration Method

```
private async Task<string> GetAIResponse(string userInput)
{
    using (var client = new HttpClient())
    {
        client.DefaultRequestHeaders.Add("Authorization", "Bearer " + apiKey);

        var requestData = new
        {
            model = "llama-3.1-8b-instant",
            messages = new[]
            {
                new { role = "user", content = userInput }
            }
        };
    }
};
```

```

        var content = new StringContent(JsonConvert.SerializeObject(requestData),
Encoding.UTF8, "application/json");

        var response = await
client.PostAsync("https://api.groq.com/openai/v1/chat/completions", content);

        var responseString = await response.Content.ReadAsStringAsync();
        dynamic result = JsonConvert.DeserializeObject(responseString);

        return result.choices[0].message.content.ToString();
    }
}

```

◆ **Button Click Event**

```

private async void BtnSend_Click(object sender, EventArgs e)
{
    string userInput = textUserInput.Text.Trim();

    textChatHistory.AppendText("You: " + userInput + Environment.NewLine);

    string botResponse = await GetAIResponse(userInput);

    textChatHistory.AppendText("Bot: " + botResponse + Environment.NewLine);

    textUserInput.Clear();
}

```

5. 🖼️ **Application Preview**

◆ **Screenshots to Include:**

- User Interface (Form design)
- Chat interaction (input/output)
- Running application

(Add screenshots here in your document)

6. Testing

Test Case Input		Output
Test 1	Hello	Greeting response
Test 2	How are you? AI generated answer	
Test 3	Empty input	Error message

7. Results & Discussion

Results

- The chatbot successfully communicates with the API
- Real-time responses are generated
- User interaction is smooth and efficient

Discussion

- The system performs well under normal conditions
 - Internet connection is required for API communication
 - API response time may vary
-

8. Limitations

- Requires active internet connection
- Depends on third-party API
- Basic UI design
- Limited customization

9. 🧠 Future Improvements

- Voice-based chatbot integration
- Improved graphical user interface
- Chat history storage
- Multi-language support
- Offline AI model support

10. 🐳 Docker Implementation

💎 Dockerfile

```
FROM mcr.microsoft.com/dotnet/sdk:7.0
```

```
WORKDIR /app
```

```
COPY . .
```

```
RUN dotnet build
```

```
CMD ["dotnet", "run"]
```

💎 Explanation

- **Base Image:** .NET SDK 7.0 for running application
- **WORKDIR:** Sets working directory
- **COPY:** Copies project files
- **RUN:** Builds application
- **CMD:** Runs application

11. 🏁 Conclusion

This project successfully demonstrates the integration of AI into a desktop application using .NET. The chatbot provides intelligent responses and enhances user interaction. It also highlights the importance of API integration and modern software development practices like Docker.

  **Author**

Muhammad Usama Bilal
